

---

# IQBRAVE

## Learning Management System

---

### *System Analysis and Evaluation Report*

|                 |                              |
|-----------------|------------------------------|
| Student Name    | H.M.S.J.Herath               |
| Course / Module |                              |
| Institution     | [ Your Campus / NVQ Centre ] |
| Submission Date | 24 March 2026                |
| Academic Year   | 2024 – 2025                  |

## TABLE OF CONTENTS

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Executive Summary .....                            | 3  |
| 2. Introduction .....                                 | 4  |
| 3. Problem Statement .....                            | 4  |
| 4. Objectives of the System .....                     | 5  |
| 5. Scope of the System .....                          | 6  |
| 5.1 In-Scope .....                                    | 6  |
| 5.2 Out-of-Scope .....                                | 6  |
| 6. System Overview .....                              | 7  |
| NVQ Multi-Tier Assessment Workflow .....              | 7  |
| 7. System Architecture .....                          | 8  |
| 7.1 MVC Layer Overview .....                          | 8  |
| Model Layer — Data & Business Logic .....             | 8  |
| View Layer — Presentation .....                       | 8  |
| Controller Layer — Routing & Direction .....          | 8  |
| 8. Technology Stack .....                             | 9  |
| 9. Module Analysis .....                              | 10 |
| 9.1 User & Authentication Management .....            | 10 |
| 9.2 Curriculum Management (Hierarchical Core) .....   | 10 |
| 9.3 Assessment & Grading Engine .....                 | 10 |
| 9.4 Verification Workflow Module (NVQ-Specific) ..... | 10 |
| 9.5 Certification & Validation Module .....           | 11 |
| 10. Database Design Overview .....                    | 12 |
| 10.1 Key Relational Structures .....                  | 12 |
| 10.2 Conceptual Entity-Relationship Diagram .....     | 12 |
| 11. API Design & Communication Flow .....             | 13 |
| 12. UI/UX Evaluation .....                            | 13 |
| 13. Security Analysis .....                           | 14 |
| 14. Testing Strategy .....                            | 14 |
| 15. Strengths & Weaknesses .....                      | 15 |
| 16. Challenges .....                                  | 15 |
| 17. Recommendations & Future Improvements .....       | 16 |
| 18. Conclusion .....                                  | 17 |
| References .....                                      | 17 |

# 1. Executive Summary

The rapid digitisation of educational platforms demands robust software architectures capable of securely managing complex learning workflows, hierarchical curricular structures, and rigorous assessment methodologies. This report provides a comprehensive architectural and functional analysis of the IQBRAVE Learning Management System — a bespoke web application engineered specifically to facilitate vocational education aligned with National Vocational Qualification (NVQ) standards.

Evaluated through a rigorous reverse-engineering methodology — examining source code, database schemas, and configuration dependencies without reliance on pre-existing functional assumptions — the IQBRAVE system is revealed as a monolithic, Model-View-Controller (MVC) web application built atop the Laravel 12 framework (PHP 8.2+).

The application leverages a utility-first CSS framework (Tailwind CSS) integrated with Alpine.js to deliver responsive, reactive front-end experiences without the overhead of heavy Single Page Application (SPA) libraries. Crucially, the system features a rigid Role-Based Access Control (RBAC) implementation distinguishing between Students, Instructors, Assessors, and Administrators, enabling a multi-tier assessment verification workflow essential for NVQ compliance.

This report delineates the architectural paradigms, technology stack rationale, module functionalities, database relationship structures, and system security implementations, culminating in strategic software engineering recommendations for future scalability and integration.

## 2. Introduction

In contemporary educational technology, Learning Management Systems (LMS) serve as the fundamental infrastructure for delivering, tracking, and assessing learning outcomes. While generic LMS platforms such as Moodle or Canvas provide broad capabilities, specialised vocational training regimens — particularly those governed by National Vocational Qualification (NVQ) frameworks — require strict adherence to competency-based assessments and multi-layered grading verifications.

This analysis investigates the IQBRAVE LMS, a custom-engineered solution designed precisely to meet these rigorous academic and vocational standards. By abstracting the system's codebase — which seamlessly integrates a sophisticated PHP backend with a modern JavaScript-enhanced front-end — this document critically evaluates how the application translates stringent real-world educational verification policies into verifiable, normalised relational data and intuitive user interfaces.

The purpose of this report is to present an academic software evaluation of the IQBRAVE LMS, outlining its systemic purpose, structural design, and operational efficacy.

## 3. Problem Statement

Vocational training and certification programmes natively demand more complex verification mechanisms than standard academic courses. Standard, off-the-shelf LMS platforms primarily operate on a simplified instructor-student dynamic: an instructor assigns a grade, and the system records it.

However, vocational frameworks require a competency-based model involving Assessors — third-party verifiers who validate the instructor's grading against national standards before a certificate can be formally issued. Developing an educational platform that natively supports this tripartite user dynamic (Student, Instructor, Assessor) while maintaining seamless enrolment, tracking, and content delivery presents a significant software engineering challenge.

The absence of standard modules for multi-tier assignment verification and built-in competency logging in generic LMS platforms forces institutions to rely on disjointed, manual, and error-prone administrative processes, risking the integrity of issued vocational qualifications.

## 4. Objectives of the System

Based on empirical evidence gathered from the system's codebase — specifically models, controllers, and database migrations — the IQBRAVE LMS fundamentally aims to achieve the following operational objectives:

1. **Hierarchical Content Delivery:** To provide a structured, scalable curricular hierarchy consisting of Courses, Modules, Units, and sequential Lessons, ensuring logical knowledge progression.
2. **Multi-Tiered Assessment & Verification:** To automate a secure workflow where Student submissions are first graded by Instructors and subsequently verified by certified Assessors before final approval is recorded.
3. **Competency Tracking:** To map student progress directly against precise NVQ competency frameworks, creating a verifiable audit trail of skill acquisition.
4. **Role-Segregated User Experience:** To deliver secure, distinct dashboard experiences tailored to the distinct operational needs of Administrators, Assessors, Instructors, and Students.
5. **Automated Credentialing:** To securely generate verifiable, cryptographically sound digital certificates fortified with embedded QR codes upon the successful completion of the multi-tier verification process.

## 5. Scope of the System

The functional boundaries of the IQBRAVE LMS encompass the complete lifecycle of a vocational student's digital learning journey.

### 5.1 In-Scope

The system manages user authentication, dynamic role assignment, course creation and enrolment, multi-format assessments (objective quizzes and subjective file-upload assignments), rigorous grading workflows, lesson progress tracking, and verifiable certificate generation. It acts as the singular source of truth for student competency records.

### 5.2 Out-of-Scope

The current iteration, based on codebase analysis, explicitly excludes:

- Built-in financial transaction processing (e.g., Stripe or PayPal integration for course purchases).
- Synchronous live-video broadcasting (e.g., WebRTC or Zoom integration).
- Enrolment and financial administration handled out-of-band or by administrative intervention.
- Live communication mechanisms such as chat forums, which are not native to the current schema.

## 6. System Overview

The IQBRAVE LMS operates as a centralised web portal where educational governance is executed via an intuitive visual interface. Administrators initialise the environment by configuring the curriculum hierarchy and authorising faculty accounts. Instructors populate courses with multimedia lessons, objective quizzes, and complex assignments. Students enrol into courses, track their lesson progression through dynamic UI indicators, and submit assessments.

The defining feature of the system is its resultant workflow: when a student submits an assignment, an Instructor issues an initial grade. If the course is flagged for NVQ compliance, this grade remains in a 'pending verification' state. An Assessor subsequently reviews the physical or digital evidence, confirms internal compliance with the competency standard, and formally validates the grade. Only upon this final verification is the student eligible to receive an auto-generated, QR-verifiable certificate as definitive proof of vocational qualification.

### NVQ Multi-Tier Assessment Workflow

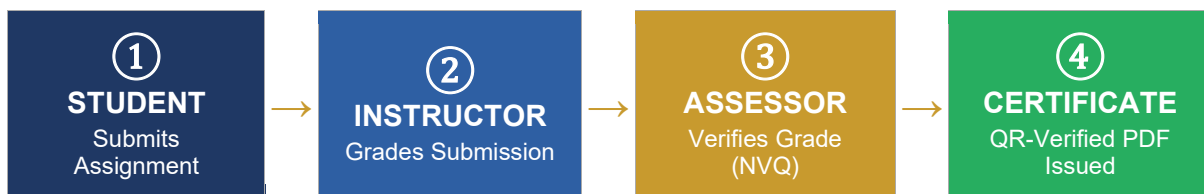


Figure 1: NVQ Assessment & Certification Workflow

## 7. System Architecture

The IQBRAVE system is architected as a Server-Side Rendered Monolithic Web Application adhering strictly to the Model-View-Controller (MVC) design pattern, natively enforced by the Laravel framework. This architectural choice prioritises developer cohesion, deployment simplicity, and maintainability over the distributed complexity of microservices.

### 7.1 MVC Layer Overview

| Layer      | Technology                              | Key Responsibility                                            |
|------------|-----------------------------------------|---------------------------------------------------------------|
| Model      | Eloquent ORM (Active Record)            | Business logic, database queries, relationship mapping        |
| View       | Blade Engine + Tailwind CSS + Alpine.js | Server-rendered HTML, reactive components, responsive layouts |
| Controller | Laravel HTTP Controllers + Middleware   | Request routing, RBAC enforcement, data aggregation for views |

Table 1: MVC Architecture Layer Decomposition

#### **Model Layer — Data & Business Logic**

Handled by Laravel's Eloquent Object-Relational Mapper (ORM). Models such as Course, User, AssignmentSubmission, and CompetencyAssessment encapsulate the business rules and database interactions using Active Record paradigms. Complex polymorphic and many-to-many relationships are elegantly managed via Eloquent pivot tables.

#### **View Layer — Presentation**

Utilising the Blade templating engine, views are compiled on the server and delivered to the client as pristine HTML. The views are modularised into reusable components, styled using Tailwind CSS utility classes, and augmented for client-side reactivity using Alpine.js, delivering a near-SPA user experience without the overhead of a detached frontend framework.

#### **Controller Layer — Routing & Direction**

HTTP requests are intercepted by the Laravel Router, verified against Middleware (for role and authentication checks), and routed to specific controllers — such as AssessorController and StudentDashboardController — which orchestrate data retrieval and view construction.



## 8. Technology Stack

The technology stack of the IQBRAVE LMS has been selected to optimise time-to-market, developer ergonomics, and system reliability while avoiding the complexity overhead of detached microservices.

| Category           | Technology / Library            | Purpose                                                                   |
|--------------------|---------------------------------|---------------------------------------------------------------------------|
| <b>Backend</b>     | PHP 8.2+ / Laravel 12           | Core framework, routing, auth, migrations                                 |
| ORM                | Eloquent ORM                    | Active Record database abstraction, pivot & polymorphic relations         |
| <b>Frontend JS</b> | Alpine.js                       | Lightweight reactivity — modals, toggles, progress bars                   |
| CSS                | Tailwind CSS                    | Utility-first responsive styling across all dashboards                    |
| <b>Build Tool</b>  | Vite                            | Ultra-fast asset bundling & Hot Module Replacement (HMR)                  |
| PDF Engine         | barryvdh/laravel-dompdf         | Server-side generation of pixel-perfect certificate PDFs                  |
| <b>QR Code</b>     | simplesoftwareio/simple-qr-code | Embeds cryptographic QR codes on certificates for credential verification |

Table 2: IQBRAVE LMS Technology Stack

## 9. Module Analysis

The system is logically partitioned into five cohesive domain modules, each addressing a specific functional concern within the educational workflow.

### 9.1 User & Authentication Management

Built atop Laravel Breeze, this module handles secure login, session management, and cryptographic password hashing (Bcrypt). An integral component is the Role-Based Access Control (RBAC) implemented via constants in the User model: `ROLE_ADMIN`, `ROLE_INSTRUCTOR`, `ROLE_ASSESSOR`, and `ROLE_STUDENT`. Route middleware guarantees strict boundary separation, ensuring that an Instructor cannot access the Assessor verification queues.

### 9.2 Curriculum Management (Hierarchical Core)

This module defines the academic structure through a deeply nested hierarchy:

- Courses — the top-level container encapsulating metadata, thumbnails, and statuses.
- Modules & Units — organisational folders grouping related concepts, enhanced with NVQ Fields to map to national standards.
- Lessons — granular instructional entities containing rich text, embedded multimedia, and associated completion tracking.

### 9.3 Assessment & Grading Engine

IQBRAVE provides dual-track assessment capabilities to support both objective and subjective evaluation:

- Quizzes — Objective, auto-graded assessments. The module manages complex relationships spanning Quiz, QuizQuestion, QuizOption, and QuizAttempt models, calculating grades instantaneously upon submission.
- Assignments — Subjective, file-based submissions utilising the Assignment, AssignmentSubmission, and AssignmentResult models, providing a structured workflow for file uploads, deadline enforcement, and instructor rubric-based grading.

### 9.4 Verification Workflow Module (NVQ-Specific)

This is the system's most intricate and distinctive module. Upon a student submitting an assignment, the status is flagged to require instructor grading. Once graded, the VerificationLog

and CompetencyAssessment models are engaged. The submission is queued in the Assessor's dashboard; the Assessor reviews the grading against external competency standards, subsequently approving or rejecting the workflow. This ensures a tamper-proof audit trail of educational validation.

Key Models: VerificationLog · CompetencyAssessment · AssignmentSubmission  
Submission States: Pending → Graded → Assessor Reviewed → Verified  
This workflow directly satisfies NVQ regulatory requirements for third-party verification.

## 9.5 Certification & Validation Module

Upon fulfilling course requirements, the system triggers the automatic generation of a Certificate record. The DOMPDF library renders a custom Blade view into a downloadable PDF format. Simultaneously, a unique QR code is mathematically generated and affixed to the document, directing scanners to an unauthenticated validation route (VerifyCertificateController), preventing credential forgery.

## 10. Database Design Overview

The IQBRAVE LMS employs a highly normalised Relational Database Management System (RDBMS) architecture, evident from the extensive suite of migration files. The schema heavily utilises Foreign Key constraints to maintain absolute referential integrity across the complex educational data models.

### 10.1 Key Relational Structures

6. Many-to-Many Assignments — The database expertly utilises pivot tables (course\_user, module\_user, enrollments) to handle dynamic assignments. A course can have many students, and a student can have many courses, mediated by the enrollments table which additionally tracks metadata such as enrolled\_at and status.
7. Polymorphic Workflows — The assignment\_submissions table acts as a central hub, containing critical status enums (pending, graded, verified) that dictate the flow of data between the disparate Controller layers (Student to Instructor to Assessor).
8. Audit Trailing — The verification\_logs and competency\_assessments tables serve an append-only audit tracking purpose, mapping back to specific submissions and assessors, providing the essential legal and academic non-repudiation required by vocational governing bodies.

### 10.2 Conceptual Entity-Relationship Diagram

| Entity A                      | Relation | Pivot / Bridge        | Relation | Entity B                     |
|-------------------------------|----------|-----------------------|----------|------------------------------|
| <b>Users (1)</b>              | ——< (M)  | Enrollments (M)       | ——> (1)  | <b>Courses</b>               |
| <b>Courses (1)</b>            | ——< (M)  | Modules (1)           | ——< (M)  | <b>Units / Lessons</b>       |
| <b>Courses (1)</b>            | ——< (M)  | Assignments (1)       | ——< (M)  | <b>AssignmentSubmissions</b> |
| <b>Users [Student] (1)</b>    | ——< (M)  | AssignmentSubmissions | ——       |                              |
| <b>Users [Instructor] (1)</b> | ——< (M)  | AssignmentResults     | ——       |                              |
| <b>Users [Assessor] (1)</b>   | ——< (M)  | VerificationLogs      | ——       |                              |

Table 3: Entity-Relationship Overview — IQBRAVE LMS Database Schema

## 11. API Design & Communication Flow

Unlike decoupled architectures that rely heavily on robust REST or GraphQL APIs, IQBRAVE operates via a classic synchronous request-response HTTP lifecycle. Forms are submitted entirely securely via POST requests encompassing CSRF (Cross-Site Request Forgery) tokens injected seamlessly by the Blade engine.

However, progressive enhancement is utilised strategically. For non-destructive, high-frequency actions — such as expanding a curriculum tree, toggling a lesson as 'completed' (LessonProgress), or validating instant quiz answers — the system relies on Axios (bundled via Vite) alongside Alpine.js directives (x-data, x-on). This hybrid approach negates the need for full page reloads for micro-interactions, vastly improving perceived performance whilst insulating the core logic entirely server-side.

## 12. UI/UX Evaluation

The User Interface is predicated on modern, clean aesthetics powered universally by Tailwind CSS. Scrutiny of the technological choices reveals the following UX characteristics:

- **Responsive Ubiquity** — The utility-first nature of Tailwind guarantees that dashboards (Admin, Instructor, Assessor, Student) are fully fluid, scaling gracefully to mobile devices. This is crucial for students accessing lessons via smartphones.
- **State-Driven Interactivity** — Alpine.js prevents DOM manipulation spaghetti code. Modals, alert notifications, and interactive lesson sidebars respond instantaneously to localised component state, minimising cognitive load without complex state-management tools such as Redux.
- **Visual Hierarchy** — The distinct dashboard controllers imply heavily customised views. An Assessor requires a dense, tabular data grid of pending verifications, whereas a Student requires a distraction-free, hyper-focused lesson consumption interface. The MVC pattern natively supports rendering entirely divergent Blade layouts based on user roles.

## 13. Security Analysis

As an educational platform maintaining Personally Identifiable Information (PII) and legally binding academic records, security is paramount. The system leverages the extensive protections afforded by Laravel 12:

9. **Authentication & Cryptography:** Passwords are mathematically hashed using Bcrypt before database insertion. User model casts enforce this boundary automatically, preventing plain-text storage.
10. **Authorisation & RBAC:** Middleware comprehensively envelopes all routes. An explicit authorisation layer prevents Privilege Escalation Vulnerabilities; a student manipulating request parameters cannot force access to an Assessor's verification route.
11. **Input Sanitation:** Controller validation requests shield the database from SQL Injection (SQLi) and Cross-Site Scripting (XSS). Laravel Eloquent utilises prepared statements exclusively, eliminating a broad attack surface.
12. **Verification Integrity:** The QR Code certificate validation ensures the system's output (credentials) cannot be forged externally, maintaining the institutional reputation and verifying academic award integrity.

## 14. Testing Strategy

To maintain the integrity of a highly complex, multi-tiered grading workflow, a rigorous multi-layered testing strategy is imperative for the IQBRAVE LMS:

- Unit Testing (PHPUnit / Pest) — Exhaustively tests isolated mathematical logic. For example, ensuring the Course->progressForStudent() helper model function accurately computes percentage completions without division-by-zero errors.
- Integration / Feature Testing — Crucial for the NVQ workflow. Automated tests programmatically simulate a student submitting an assignment, verify the database state shifts, authenticate as an instructor, grade the assignment, authenticate as an assessor, and verify. Any breakage in this chain is a catastrophic domain failure.
- Browser / E2E Testing (Laravel Dusk) — Simulates the intricate JavaScript-driven interactions (Alpine.js), ensuring modals trigger correctly and Axios requests appropriately update UI states upon lesson completion.

## 15. Strengths & Weaknesses

The following analysis presents a balanced evaluation of the IQBRAVE LMS's architectural and functional qualities:

| ✓ STRENGTHS                                                                       | ✗ WEAKNESSES                                                                |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Domain Specificity — uniquely addresses NVQ multi-tier verification workflows     | Synchronous Bottlenecks — PDF generation degrades performance at scale      |
| Architectural Simplicity — cohesive monolith, easier to deploy and maintain       | Monolithic Scaling — entire application must be replicated (not per-module) |
| Robust ORM — Eloquent pivot tables provide granular role & permission flexibility | LTI Non-Compliance — restricts integration with wider university ecosystems |

Table 4: Strengths and Weaknesses Summary

## 16. Challenges

The primary challenge inherited by this architectural design is State Complexity Management. The lifecycle of an AssignmentSubmission (Pending → Graded → Verified) requires absolute transactional integrity. If a database failure occurs mid-verification, the application risks entering corrupt logical states — for example, an assignment marked as verified without a corresponding VerificationLog entry.

Additionally, managing the UX density for the Assessor dashboard — presenting potentially hundreds of pending student assignments alongside grading rubrics and competency mapping — necessitates meticulous front-end design to prevent data overload and user fatigue.

## 17. Recommendations & Future Improvements

The following strategic recommendations are proposed to enhance system performance, scalability, and interoperability:

13. **Asynchronous Job Processing:** Immediately delegate PDF Certificate generation, Email Notifications, and heavy database analytical queries to Laravel's robust Queue system (e.g., utilising Redis). This will drastically improve the perceptual speed of the application during high-load scenarios.
14. **API Extraction & Mobile Application:** While the current web-responsiveness is excellent, vocational users often require robust, offline-capable mobile applications. Gradually extracting the business logic into a formalised RESTful or GraphQL API via Laravel Sanctum would pave the way for a dedicated Flutter or React Native mobile client.
15. **Implement Advanced Caching Layer:** To alleviate database pressure, query responses — especially for static curricular hierarchies (Courses and Modules) — should be aggressively cached utilising Redis or Memcached directly within the Eloquent layer.
16. **Learning Tools Interoperability (LTI) Integration:** To transition IQBRAVE from an isolated system to an enterprise-grade academic tool, implementing LTI standards would allow disparate educational tools and third-party content providers to securely integrate with the LMS.



## 18. Conclusion

In conclusion, the IQBRAVE Learning Management System represents an advanced, purpose-built educational software platform exceptionally tailored to the rigorous demands of vocational and NVQ education. The technical adoption of a Laravel 12 MVC monolithic architecture, augmented by Alpine.js and Tailwind CSS, provides a highly pragmatic, secure, and maintainable foundation.

By natively accommodating a complex, multi-tiered assessment and verification workflow that delineates Instructors and Assessors, IQBRAVE effectively bridges the gap between digital convenience and strict vocational oversight. While the architecture possesses typical monolithic constraints regarding horizontal scaling, implementing strategic refinements such as asynchronous queue processing and API extraction will fortify the system's longevity.

Ultimately, IQBRAVE stands as a highly competent, well-engineered, and academically sound digital learning solution suitable for rigorous, real-world institutional deployment. With the recommended enhancements in place, the system is well-positioned to evolve into a fully enterprise-grade, standards-compliant platform for vocational education.

## References

1. Laravel Framework. (2024). Laravel 12 Documentation. <https://laravel.com/docs/12.x>
2. Tailwind Labs. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs>
3. Alpine.js. (2024). Alpine.js Documentation. <https://alpinejs.dev>
4. Barryvdh. (2024). Laravel DomPDF Package. <https://github.com/barryvdh/laravel-dompdf>
5. PHP Group. (2024). PHP 8.2 Manual. <https://www.php.net/manual/en/>
6. National Vocational Qualifications Framework. (2024). Ofqual Regulatory Framework. <https://www.gov.uk/government/organisations/ofqual>

— End of Report —